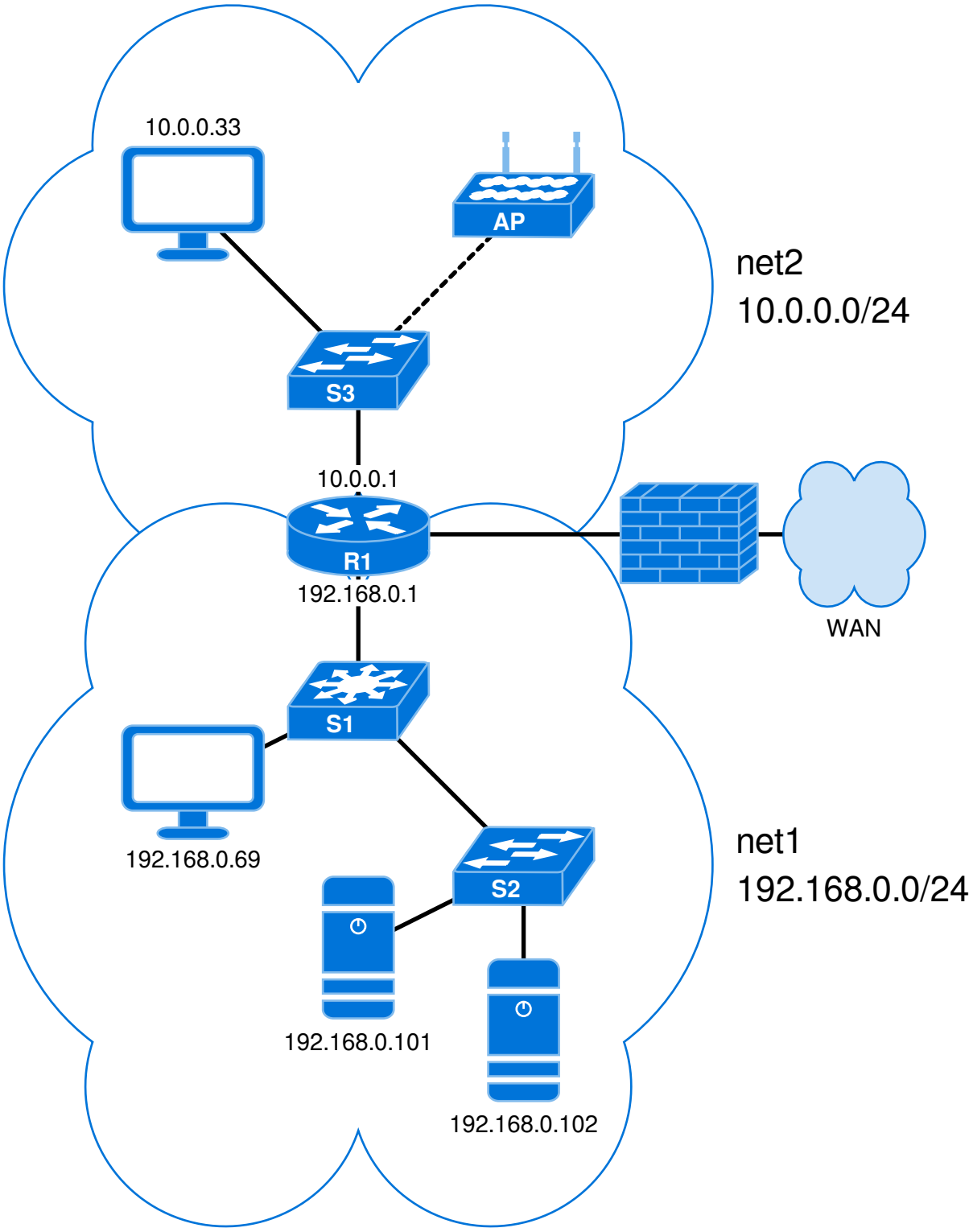
cntopo 0.1.0

A modular set of CeTZ-powered computer network topology icons with Fletcher integration.



Contents

Overview . . . 3
- Note . . . 3

API . . . 4
- Main . . . 4
  - fletcher-shapes . . . 4
  - icons . . . 4
  - to-fletcher-shapes . . . 5
  - default-nodes . . . 6
  - icon-presets . . . 6
- Node shapes . . . 6
  - node-shape-circle . . . 7
  - node-shape-hex . . . 7
  - node-shape-square . . . 7
  - node-shape-rect . . . 8
  - node-shape-bridge . . . 8
  - node-shape-firewall . . . 8
  - node-shapes . . . 8
- Node classes . . . 8
  - node-class-router . . . 9
  - node-class-switch . . . 9
  - node-class-hub . . . 10
  - node-class-fe-hub . . . 10
  - node-class-l3-switch . . . 10
  - node-class-ap . . . 11
  - node-class-dual-ap . . . 11
  - node-class-mesh-ap . . . 11
  - node-classes . . . 12
- Node details . . . 12
  - node-detail-secure . . . 12
  - node-detail-cloud . . . 13
  - node-details . . . 13
- Nodes . . . 13
  - node . . . 14
- Clients . . . 16
  - monitor . . . 17
  - server . . . 17
- Miscellaneous . . . 18
  - lock . . . 18
  - cloud . . . 19
- Shapes . . . 19
  - antennas . . . 19
  - arrow . . . 20
  - arrows . . . 21
  - wireless-wave . . . 22

Shapes . . . 23

Overview

`cntopo-typ` is a collection of computer network topology icons, designed to be easy to use, extensible and work well with existing frameworks like CeTZ or Fletcher. The two main functions it provides are `fletcher-shapes` and `icons`, yielding all of the icons for use with Fletcher and CeTZ respectively. Both are parametarized the same way, as well as most of the icons in this package. All `node` icons are provided with 2d (`flat: true`) and 3d (`flat: false`) variants.

Some important terms to clarify how the icon construction works:

- Node: An icon representing a node in a network which is not a client
- Client: An icon representing a client node in a network
- Shape: The container holding the node's characteristics (icon, name, details…), eg. circle
- Class: The large portion of a node containing an icon or content
- Detail: The small bottom portion of a node containing an icon or content

Note

Whenever `..pos` is used, the positional arguments can be one of:

- none ⇒ defaults to the position `(0,0)` and a preconfigured default size for the specific shape
- `(x, y)` ⇒ sets the position of the icon to the x and y value
- `(x, y), (w, h)` ⇒ sets the position of the icon to the x and y value and its radius to w (x radius) and h (y radius)
- `(x, y), r` ⇒ sets the radius of the icon to `(r, r)`

It is recommended to explicitly set the `node` sizes when working with Fletcher like so:

```
#import "@preview/fletcher:0.5.8": diagram, edge, node

#let node = node.with(width: 4em, height: 4em)

#diagram( ... )
```

API

Main

- fletcher-shapes()
- icons()
- to-fletcher-shapes()

Variables

- default-nodes
- icon-presets

fletcher-shapes

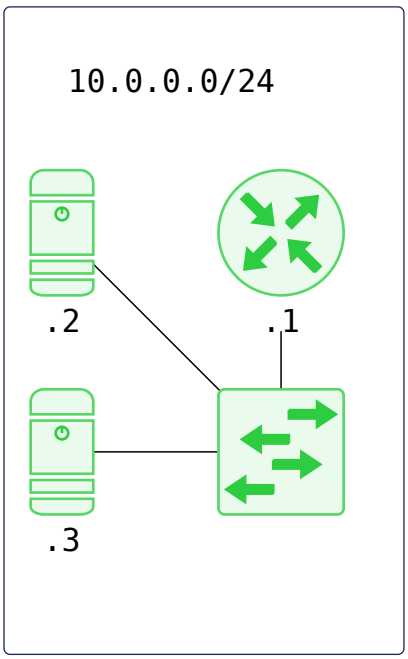
fletcher-shapes(..args: `arguments`) -> `dict`

Shapes to use in Fletcher diagrams.

```
#import "@preview/fletcher:0.5.8": diagram, edge, node

#let node = node.with(width: 4em, height: 4em)
#let (router, switch, server) = fletcher-shapes(
  stroke: green.lighten(20%),
  fill: green.lighten(90%),
  stroke-inner: green,
  fill-inner: green
)

#diagram(
  node((0.5,0.25), [10.0.0.0/24]),
  node((0,1), shape: server.with(label: ".2")),
  edge(),
  node((1,2), shape: switch, name: <s>),
  edge(),
  node((0,2), shape: server.with(label: ".3")),
  node((1,1), shape: router.with(label: ".1")),
  edge(<s>),
)
```



Parameters:

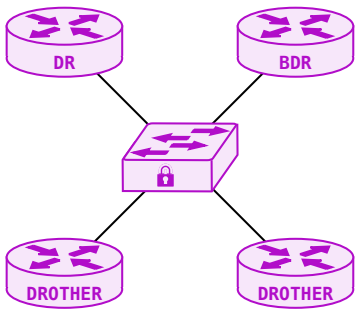
`..args` (`arguments`) – The same args as in `icons()`

icons

icons(
  stroke: `stroke`,
  fill: `paint`,
  stroke-inner: `stroke` `auto`,
  fill-inner: `paint` `auto`,
  flat: `bool`
) -> `dict`

Applies styling to all of the CeTZ icons

```
#let (router, switch, l3-switch, cloud) = icons(
  stroke: purple,
  fill: purple.lighten(90%),
  flat: false
)
#set text(size: .75em)
#canvas({
  import cetz.draw: *
  line((2,2), (0,4))
  line((2,2), (4,4))
  line((2,2), (0,0))
  line((2,2), (4,0))
  router((0,4), detail: [DR])
  router((4,4), detail: [BDR])
  router((0,0), detail: [DROTHER])
  router((4,0), detail: [DROTHER])
  switch((2,2), detail: "secure")
})
```



Parameters:

`stroke` (`stroke` = `stroke-def`) – Icon outer stroke

`fill` (`paint` = `fill-def`) – Icon main fill

`stroke-inner` (`stroke` or `auto` = `auto`) – Icon inner stroke, defaults to `icons.stroke`

`fill-inner` (`paint` or `auto` = `auto`) – Icon inner fill, defaults to paint of `icons.stroke`

`flat` (`bool` = `true`) – If the icon should be flat or 3d

to-fletcher-shapes

to-fletcher-shapes(i: `dict`) -> `dict`

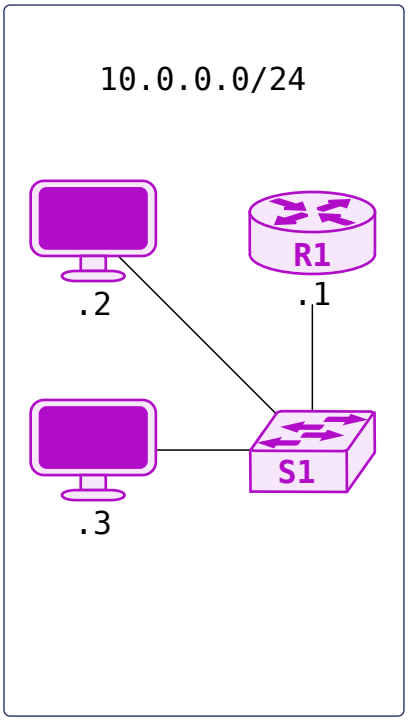
Helper function to convert the CeTZ icons to Fletcher shapes

```
#import "@preview/fletcher:0.5.8": diagram, edge, node

#let i = icons(
  stroke: purple,
  fill: purple.lighten(90%),
  flat: false
)
#let (router, monitor, switch) = to-fletcher-shapes(i)
#let node = node.with(width: 4em, height: 4em)

#diagram(
  node((0.5,0.25), [10.0.0.0/24]),
  node((0,1), shape: monitor.with(label: ".2")),
  edge(),
  node((1,2), shape: switch.with(detail: [S1]), name:
<s>),
  edge(),
  node((0,2), shape: monitor.with(label: ".3")),
  node((1,1), shape: router.with(label: ".1", detail:
```

```
[R1])),
  edge(<s>),
)
```



Parameters:

`i` (`dict`) – CeTZ icons

default-nodes `dict`

Default node icons

icon-presets `dict`

Icon presets

These shapes will be available from the `icons()` and `fletcher-shapes()` functions. A complete showcase of all of the icons can be found at the end of this document

Node shapes

All node shapes can be accessed in these three ways:

```
// Example: rect
node(shape: node-shape-rect)
node(shape: node-shapes.rect)
node(shape: "rect")           // -> recommended
```

A custom node shape can be passed to the `shape` parameter. It must return a cetz object and conform to the following signature:

```
#let my-custom-shape = (
  sx,     /// width
```

```
  sy,     /// height
  radius, /// corner radius
  stroke, /// stroke
  fill,   /// fill
  flat    /// if shape is 3d
) => { ... }
```

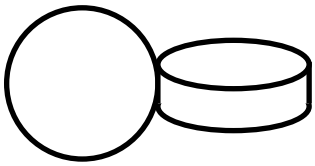
Variables

- node-shape-circle
- node-shape-hex
- node-shape-square
- node-shape-rect
- node-shape-bridge
- node-shape-firewall
- node-shapes

node-shape-circle `function`

Circular node shape

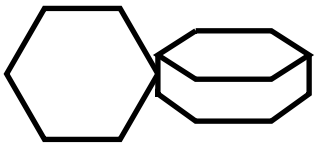
```
#cetz.canvas({
  node((0,0), shape: "circle")
  node((2,0), shape: "circle", flat: false)
})
```



node-shape-hex `function`

Hexagonal node shape

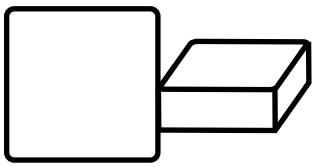
```
#cetz.canvas({
  node((0,0), shape: "hex")
  node((2,0), shape: "hex", flat: false)
})
```



node-shape-square `function`

Square node shape

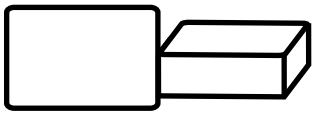
```
#cetz.canvas({
  node((0,0), shape: "square")
  node((2,0), shape: "square", flat: false)
})
```



node-shape-rect `function`

Rectangular node shape

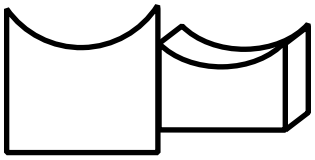
```
#cetz.canvas({
  node((0,0), shape: "rect")
  node((2,0), shape: "rect", flat: false)
})
```



node-shape-bridge `function`

Bridge node shape

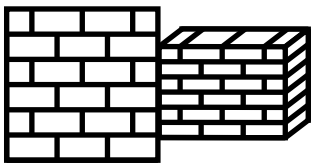
```
#cetz.canvas({
  node((0,0), shape: "bridge")
  node((2,0), shape: "bridge", flat: false)
})
```



node-shape-firewall `function`

Firewall node shape

```
#cetz.canvas({
  node((0,0), shape: "firewall")
  node((2,0), shape: "firewall", flat: false)
})
```



node-shapes `dict`

Dict containing all of the node shape presets

Node classes

All node classes can be accessed in these three ways:

```
// Example: router
node(class: node-class-router)
node(class: node-classes.router)
node(class: "router")         // -> recommended
```

A custom node class can be passed to the `class` parameter. It must return a cetz object and conforort to the following signature:

```
#let my-custom-class = (
  sx,     /// parent node width
  sy,     /// parent node height
  stroke, /// stroke
  fill    /// fill
) => { ... }
```

Classes can also be regular content:

```
node(class: [ABC])
```

Classes can also be used as details and vice-versa.

NOTE: Classes influence the node's `shape` if it's set to `auto`

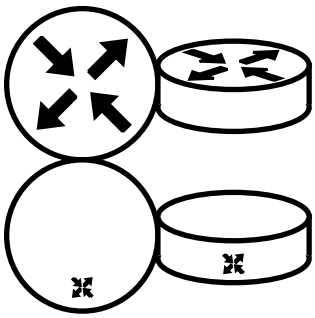
Variables

- node-class-router
- node-class-switch
- node-class-hub
- node-class-fe-hub
- node-class-l3-switch
- node-class-ap
- node-class-dual-ap
- node-class-mesh-ap
- node-classes

node-class-router `function`

Router node class

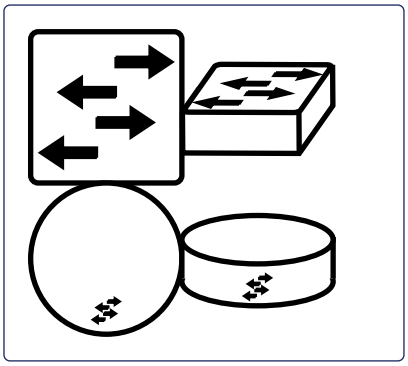
```
#cetz.canvas({
  node((0,2), class: "router")
  node((2,2), class: "router", flat: false)
  node((0,0), detail: "router")
  node((2,0), detail: "router", flat: false)
})
```



node-class-switch `function`

Switch node class

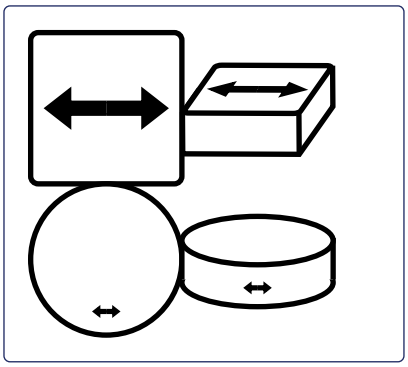
```
#cetz.canvas({
  node((0,2), class: "switch")
  node((2,2), class: "switch", flat: false)
  node((0,0), detail: "switch")
  node((2,0), detail: "switch", flat: false)
})
```



node-class-hub `function`

Hub node class

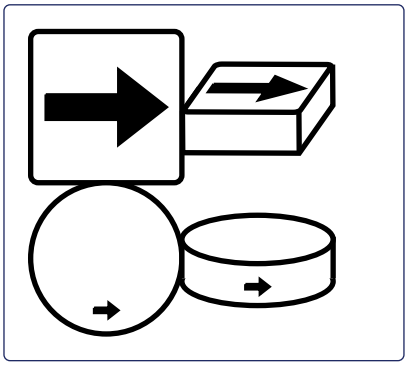
```
#cetz.canvas({
  node((0,2), class: "hub")
  node((2,2), class: "hub", flat: false)
  node((0,0), detail: "hub")
  node((2,0), detail: "hub", flat: false)
})
```



node-class-fe-hub `function`

Fast ethernet hub node class

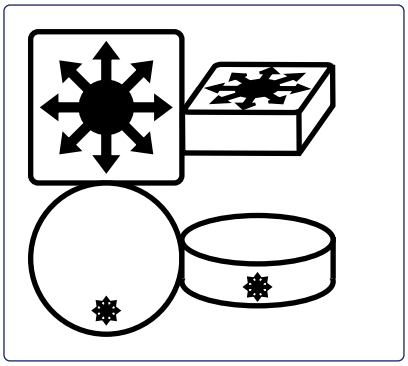
```
#cetz.canvas({
  node((0,2), class: "fe-hub")
  node((2,2), class: "fe-hub", flat: false)
  node((0,0), detail: "fe-hub")
  node((2,0), detail: "fe-hub", flat: false)
})
```



node-class-l3-switch `function`

Layer 3 switch node class

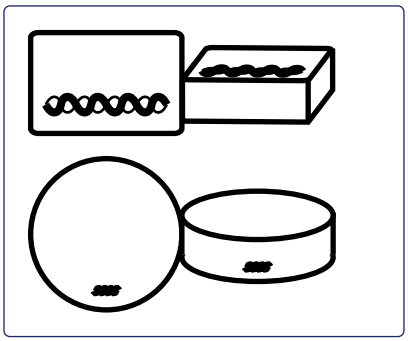
```
#cetz.canvas({
  node((0,2), class: "l3-switch")
  node((2,2), class: "l3-switch", flat: false)
  node((0,0), detail: "l3-switch")
  node((2,0), detail: "l3-switch", flat: false)
})
```



node-class-ap `function`

Access point node class

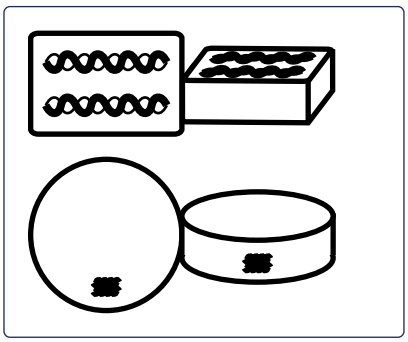
```
#cetz.canvas({
  node((0,2), class: "ap")
  node((2,2), class: "ap", flat: false)
  node((0,0), detail: "ap")
  node((2,0), detail: "ap", flat: false)
})
```



node-class-dual-ap `function`

Dual access point node class

```
#cetz.canvas({
  node((0,2), class: "dual-ap")
  node((2,2), class: "dual-ap", flat: false)
  node((0,0), detail: "dual-ap")
  node((2,0), detail: "dual-ap", flat: false)
})
```

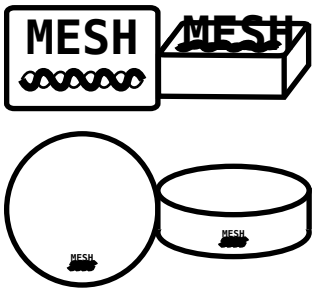


node-class-mesh-ap `function`

Mesh access point node class

As you can see, the text transformation doesn't work yet in 3d

```
#cetz.canvas({
  node((0,2), class: "mesh-ap")
  node((2,2), class: "mesh-ap", flat: false)
  node((0,0), detail: "mesh-ap")
  node((2,0), detail: "mesh-ap", flat: false)
})
```



node-classes `dict`

Dict containing all of the node class presets

Node details

All node details can be accessed in these three ways:

```
// Example: secure
node(detail: node-detail-secure)
node(detail: node-details.secure)
node(detail: "secure")        // -> recommended
```

A custom node detail can be passed to the detail parameter. It must return a cetz object and conform to the following signature:

```
#let my-custom-detail = (
  sx,     /// parent node width
  sy,     /// parent node height
  stroke, /// stroke
  fill    /// fill
) => { ... }
```

Just like classes, details can also be regular content:

```
node(detail: [ABC])
```

Variables

- node-detail-secure
- node-detail-cloud
- node-details

node-detail-secure `function`

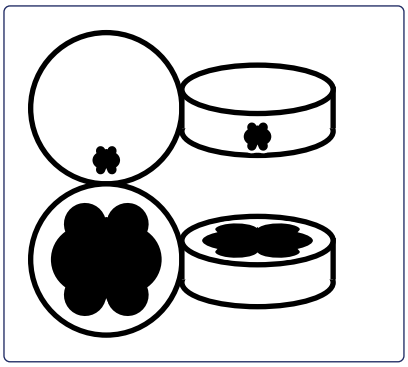
Secure node detail

```
#cetz.canvas({
  node((0,2), detail: "secure")
  node((2,2), detail: "secure", flat: false)
  node((0,0), class: "secure")
  node((2,0), class: "secure", flat: false)
})
```

node-detail-cloud `function`

Cloud node detail

```
#cetz.canvas({
  node((0,2), detail: "cloud")
  node((2,2), detail: "cloud", flat: false)
  node((0,0), class: "cloud")
  node((2,0), class: "cloud", flat: false)
})
```



node-details `dict`

Dict containing all of the node detail presets

Nodes

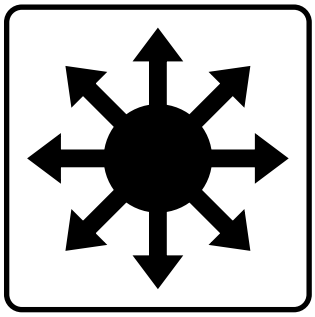
- node()

node

```
node(
  ..pos: (x, y) (x, y), (w, h),
  stroke: stroke,
  fill: paint,
  stroke-inner: stroke auto,
  fill-inner: paint auto,
  flat: bool,
  label: str content none,
  label-pos: alignment,
  detail: "secure" "cloud" content none,
  class: "router" "switch" "hub" "fe-hub" "l3-switch" "ap" "dual-ap" "mesh-ap" content none,
  shape: "circle" "hex" "square" "rect" "bridge" "firewall" auto,
  radius: int float relative auto,
  wireless: bool
)
```

Node

```
#cetz.canvas({
  node((0,0), (2,2), class: "l3-switch")
})
```



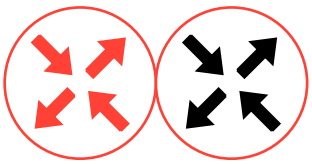
Parameters:

`..pos` (`(x, y)` or `(x, y), (w, h)`) – The position (and size)

`stroke` (`stroke` = `stroke-def`) – Icon outer stroke

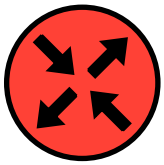
If `node.stroke-inner` and/or `node.fill-inner` are set to `auto`, then this will apply to them as well.

```
#cetz.canvas({
  node((0,0), class: "router", stroke: red)
  node((2,0), class: "router", stroke: red, stroke-inner:
black, fill-inner: black)
})
```



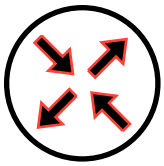
`fill` (`paint` = `fill-def`) – Icon main fill

```
#cetz.canvas({
  node(fill: red, class: "router")
})
```



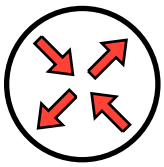
`stroke-inner` (`stroke` or `auto` = `auto`) – Icon inner stroke

```
#cetz.canvas({
  node(stroke-inner: red, class: "router")
})
```



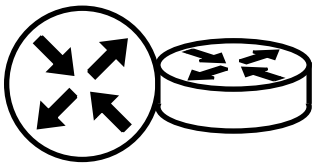
`fill-inner` (`paint` or `auto` = `auto`) – Icon inner fill

```
#cetz.canvas({
  node(fill-inner: red, class: "router")
})
```



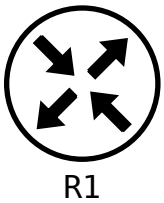
`flat` (`bool` = `true`) – If the icon should be flat or 3d

```
#cetz.canvas({
  node((0,0), flat: true, class: "router")
  node((2,0), flat: false, class: "router")
})
```



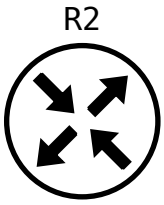
`label` (`str` or `content` or `none` = `none`) – Label

```
#cetz.canvas({
  node(label: "R1", class: "router")
})
```



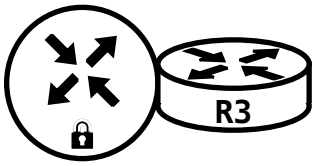
`label-pos` (`alignment` = `bottom`) – Label position

```
#cetz.canvas({
  node(label: "R2", label-pos: top, class: "router")
})
```



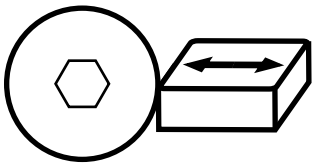
`detail` (`"secure"` or `"cloud"` or `content` or `none` = `none`) – Secondary node icon / text

```
#cetz.canvas({
  node((0,0), detail: "secure", class: "router")
  node((2,0), detail: [R3], flat: false, class: "router")
})
```



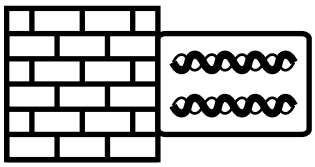
`class` (`"router"` or `"switch"` or `"hub"` or `"fe-hub"` or `"l3-switch"` or `"ap"` or `"dual-ap"` or `"mesh-ap"` or `content` or `none` = `none`) – Main node icon

```
#cetz.canvas({
  node(class: polygon.regular(size: 20pt, vertices: 6))
  node((2,0), class: "hub", flat: false)
})
```



`shape` (`"circle"` or `"hex"` or `"square"` or `"rect"` or `"bridge"` or `"firewall"` or `auto` = `auto`) – Node shape

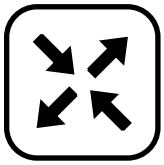
```
#cetz.canvas({
  node(shape: "firewall")
  node((2, 0), class: "dual-ap", shape: auto)
})
```



`radius` (`int` or `float` or `relative` or `auto` = `auto`) – Node radius.

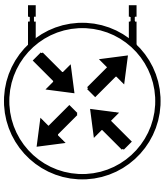
Only works on the shapes `square` or `rect`

```
#cetz.canvas({
  node(shape: "square", radius: 20%, class: "router")
})
```



`wireless` (`bool` = `false`) – If the wireless antennas should be added

```
#cetz.canvas({
  node(wireless: true, class: "router")
})
```



Clients

- monitor()
- server()

monitor

```
monitor(
  ..pos: (x, y) (x, y), (w, h),
  stroke: stroke,
  fill: paint,
  stroke-inner: stroke auto,
  fill-inner: paint auto,
  flat: bool,
  label: str content none,
  label-pos: alignment
)
```

Monitor

```
#cetz.canvas({
  monitor(label: "My PC")
})
```

Parameters:

`..pos` (`(x, y)` or `(x, y), (w, h)`) – The position (and size)

`stroke` (`stroke` = `stroke-def`) – Icon outer stroke

`fill` (`paint` = `fill-def`) – Icon main fill

`stroke-inner` (`stroke` or `auto` = `auto`) – Icon inner stroke

`fill-inner` (`paint` or `auto` = `auto`) – Icon inner fill

`flat` (`bool` = `true`) – Not implemented yet

`label` (`str` or `content` or `none` = `none`) – Label

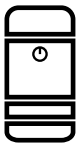
`label-pos` (`alignment` = `bottom`) – Label position

server

```
server(
  ..pos: (x, y) (x, y), (w, h),
  stroke: stroke,
  fill: paint,
  stroke-inner: stroke auto,
  fill-inner: paint auto,
  flat: bool,
  label: str content none,
  label-pos: alignment
)
```

Server

```
#cetz.canvas({
  server(label: "Some greek god's name")
})
```



Parameters:

`..pos` (`(x, y)` or `(x, y), (w, h)`) – The position (and size)

`stroke` (`stroke` = `stroke-def`) – Icon outer stroke

`fill` (`paint` = `fill-def`) – Icon main fill

`stroke-inner` (`stroke` or `auto` = `auto`) – Icon inner stroke

`fill-inner` (`paint` or `auto` = `auto`) – Icon inner fill

`flat` (`bool` = `true`) – Not implemented yet

`label` (`str` or `content` or `none` = `none`) – Label

`label-pos` (`alignment` = `bottom`) – Label position

Miscellaneous

- lock()

Variables

- cloud

lock

```
lock(
  ..pos: (x, y) (x, y), (w, h),
  stroke: stroke,
  fill: paint,
  stroke-inner: stroke auto,
  fill-inner: paint auto,
  flat: bool,
  label: str content none,
  label-pos: alignment
)
```

Lock

```
#cetz.canvas({
  lock(label: "Don't even try")
})
```

Parameters:

`..pos` (`(x, y)` or `(x, y), (w, h)`) – The position (and size)

`stroke` (`stroke` = `stroke-def`) – Icon outer stroke

`fill` (`paint` = `fill-def`) – Icon main fill

`stroke-inner` (`stroke` or `auto` = `auto`) – Icon inner stroke

`fill-inner` (`paint` or `auto` = `auto`) – Icon inner fill

`flat` (`bool` = `true`) – Not implemented yet

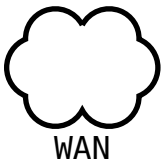
`label` (`str` or `content` or `none` = `none`) – Label

`label-pos` (`alignment` = `bottom`) – Label position

cloud

Cloud

```
#cetz.canvas({
  cloud(label: "WAN")
})
```



Shapes

- antennas()
- arrow()
- arrows()
- wireless-wave()

antennas

antennas(..pos: `(x, y)` `(x, y), (w, h)`, spacing: `int` `length` `auto`, stroke: `stroke`, fill: `paint` `auto`)

Antennas shape

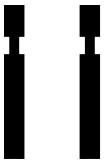
```
#cetz.canvas({
  antennas((0,0), (3,3))
})
```

Parameters:

`..pos` (`(x, y)` or `(x, y), (w, h)`) – The position (and size)

`spacing` (`int` or `length` or `auto` = `auto`) – Spacing between the antennas

```
#cetz.canvas({
  antennas((0,0), (3,3), spacing: 1)
})
```



`stroke` (`stroke` = `stroke-def`) – Antennas stroke

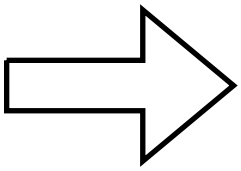
`fill` (`paint` or `auto` = `auto`) – Antennas fill

arrow

```
arrow(
  ..pos: (x, y) (x, y), (w, h),
  dir: direction,
  connect: bool,
  ratio-len: float,
  ratio-width: float,
  stroke: stroke,
  fill: paint
)
```

Arrow shape

```
#cetz.canvas({
  arrow((0,0), (3,2))
})
```



Parameters:

`..pos` (`(x, y)` or `(x, y), (w, h)`) – The position (and size)

`dir` (`direction` = `ltr`) – Direction the arrow points to

```
#cetz.canvas({
  arrow(dir: ttb)
})
```

`connect` (`bool` = `true`) – Whether to connect the tail of the arrow

```
#cetz.canvas({
  arrow(connect: false)
})
```

`ratio-len` (`float` = `3 / 5`) – Tail length to head length ratio

```
#cetz.canvas({
  arrow(ratio-len: 1 / 3)
})
```

`ratio-width` (`float` = `1 / 3`) – Tail width to head width ratio

```
#cetz.canvas({
  arrow(ratio-width: 2 / 3)
})
```

`stroke` (`stroke` = `stroke-def`) – Arrow stroke

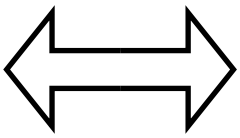
`fill` (`paint` = `fill-def`) – Arrow fill

arrows

```
arrows(
  ..pos: (x, y) (x, y), (w, h),
  type: "cross" "two" "two-v",
  ratio-len: float,
  ratio-width: float,
  stroke: stroke,
  fill: paint
)
```

Arrows shape

```
#cetz.canvas({
  arrows((0,0), (3,3))
})
```



Parameters:

`..pos` (`(x, y)` or `(x, y), (w, h)`) – The position (and size)

`type` (`"cross"` or `"two"` or `"two-v"` = `"two"`) – How arrows should be composed

```
#cetz.canvas({
  arrows((0,0), type: "two", stroke: red)
  arrows((1,0), type: "two-v", stroke: green)
  arrows((2,0), type: "cross", stroke: blue)
})
```

`ratio-len` (`float` = `3 / 5`) – Tail length to head length ratio

```
#cetz.canvas({
  arrows(ratio-len: 1 / 3)
})
```

`ratio-width` (`float` = `1 / 3`) – Tail width to head width ratio

```
#cetz.canvas({
  arrows(ratio-width: 2 / 3)
})
```

`stroke` (`stroke` = `stroke-def`) – Arrow stroke

`fill` (`paint` = `fill-def`) – Arrow fill

wireless-wave

wireless-wave(..pos: `(x, y)` `(x, y), (w, h)`, stroke: `stroke`, stroke-inner: `stroke`, n: `int`)

Wireless wave shape

```
#cetz.canvas({
  wireless-wave((0,0), (2,0.2))
})
```

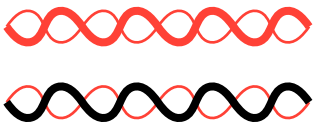
Parameters:

`..pos` (`(x, y)` or `(x, y), (w, h)`) – The position (and size)

`stroke` (`stroke` = `stroke-def`) – Wave stroke

If `inner-stroke` is set to auto, the same paint gets applied

```
#cetz.canvas({
  wireless-wave((0,1), (2,0.2), stroke: red)
  wireless-wave((0,0), (2,0.2), stroke: red, stroke-
inner: black + 3pt)
})
```



`stroke-inner` (`stroke` = `auto`) – Inner wave stroke

```
#cetz.canvas({
  wireless-wave((0,0), (2,0.2), stroke-inner: blue + 5pt)
})
```

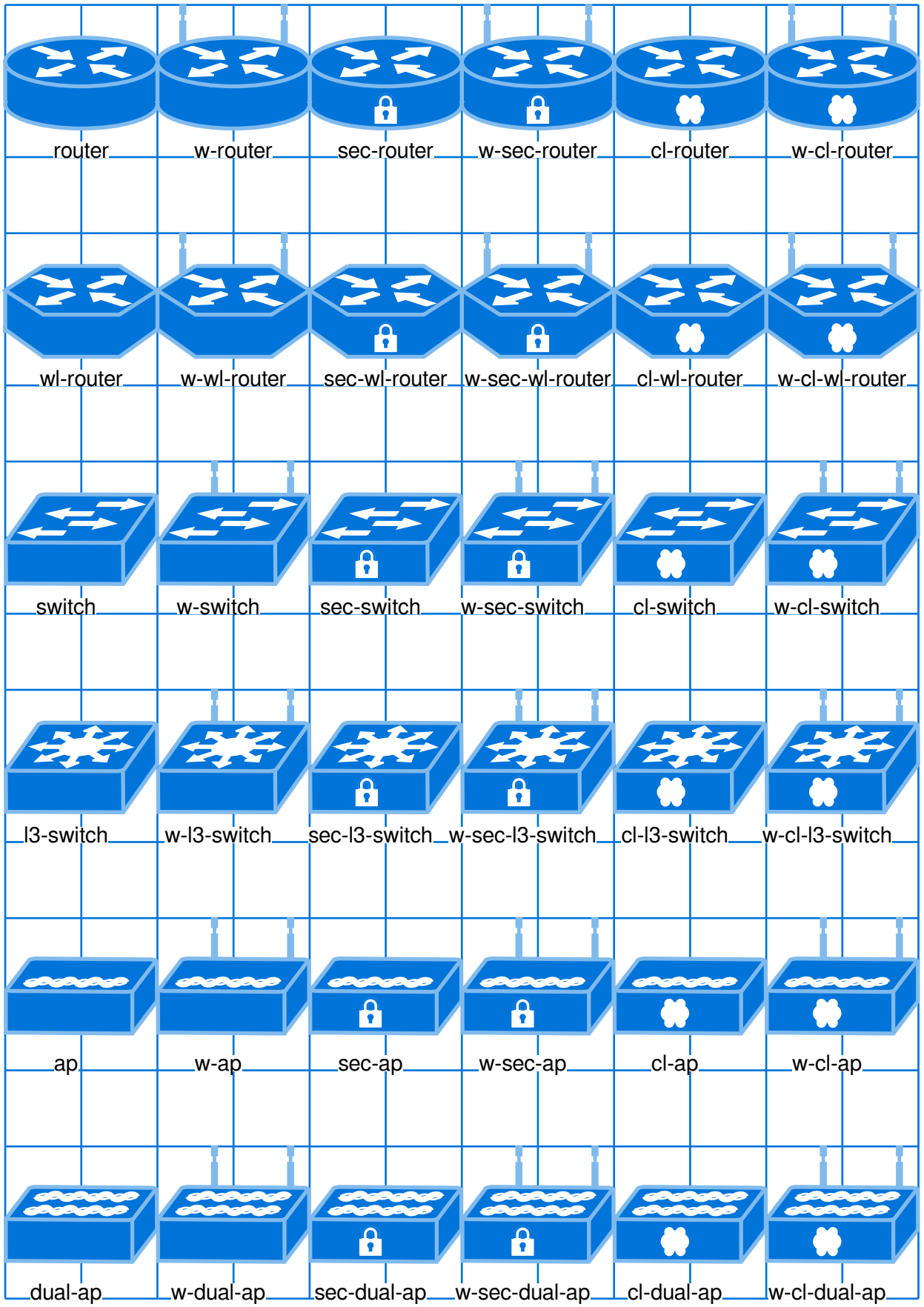
`n` (`int` = `4`) – Number of oscillations

```
#cetz.canvas({
  wireless-wave((0,0), (3,0.2), n: 12)
})
```

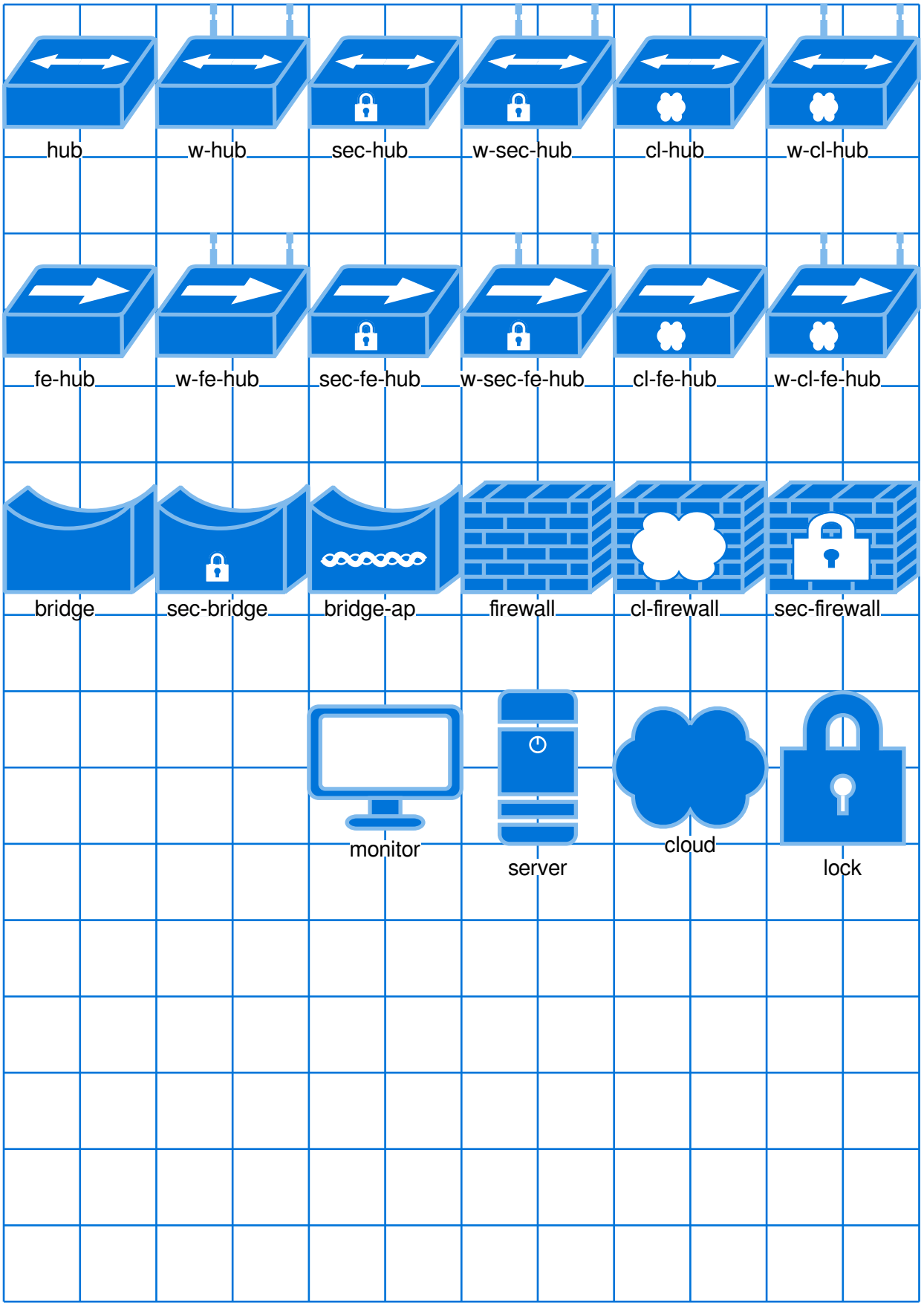
```
flat: true
```

Shapes

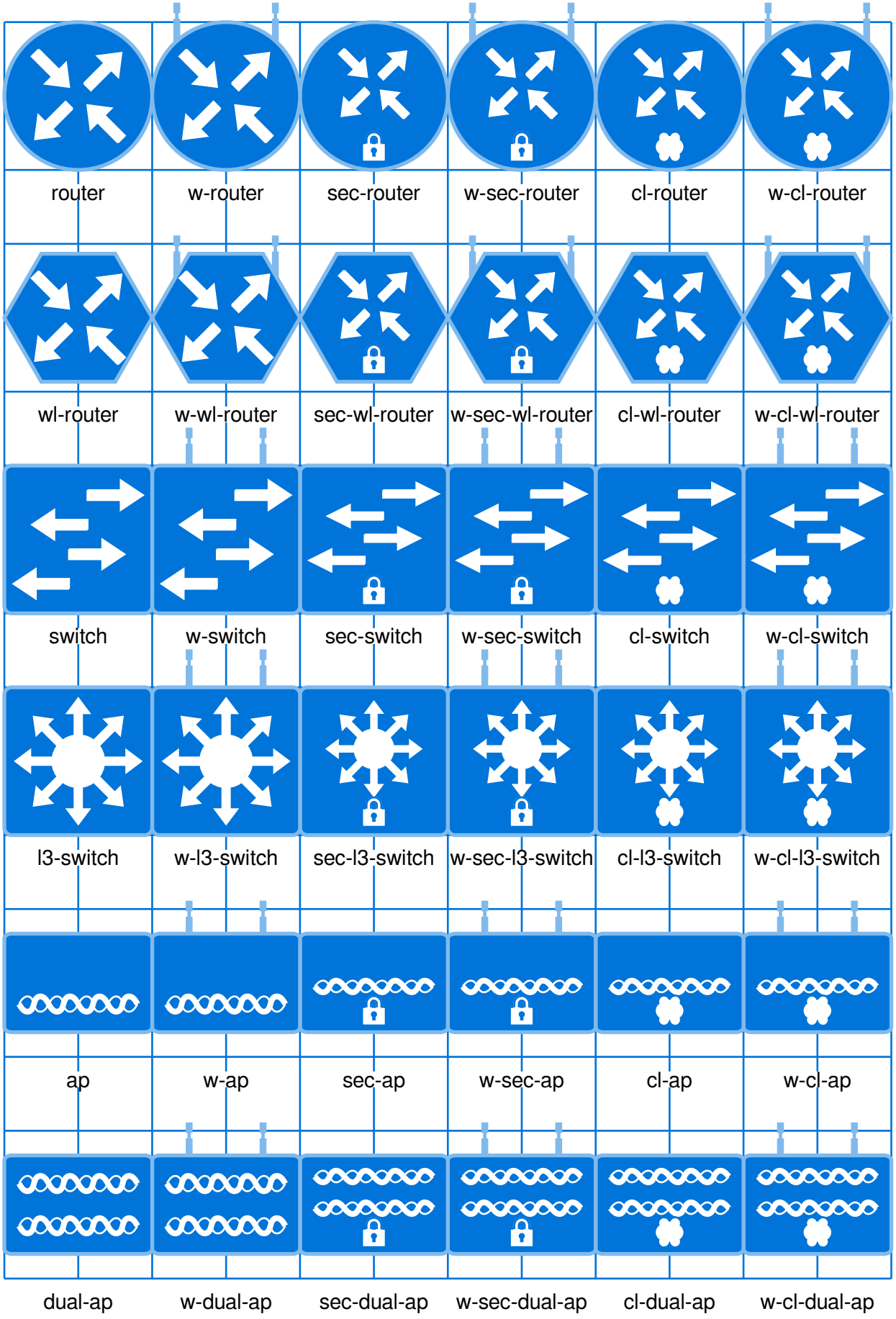
These are all of the shapes provided by `icon-presets`



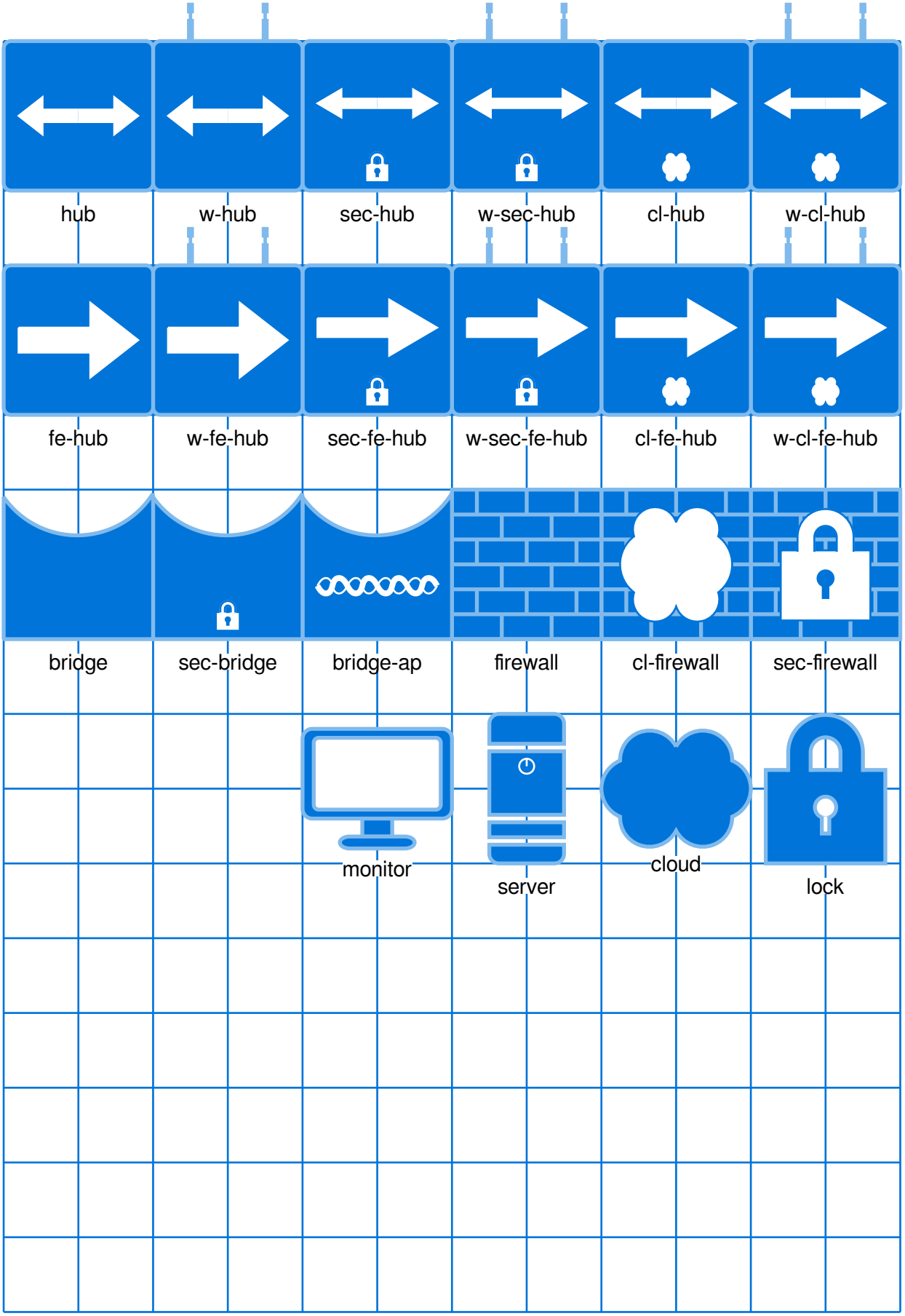
flat: true


hub
w-hub
sec-hub
w-sec-hub
cl-hub
w-cl-hub
fe-hub
w-fe-hub
sec-fe-hub
w-sec-fe-hub
cl-fe-hub
w-cl-fe-hub
bridge
sec-bridge
bridge-ap
firewall
cl-firewall
sec-firewall
monitor
server
cloud
lock

flat: false


router
w-router
sec-router
w-sec-router
cl-router
w-cl-router
wl-router
w-wl-router
sec-wl-router
w-sec-wl-router
cl-wl-router
w-cl-wl-router
switch
w-switch
sec-switch
w-sec-switch
cl-switch
w-cl-switch
l3-switch
w-l3-switch
sec-l3-switch
w-sec-l3-switch
cl-l3-switch
w-cl-l3-switch
ap
w-ap
sec-ap
w-sec-ap
cl-ap
w-cl-ap
dual-ap
w-dual-ap
sec-dual-ap
w-sec-dual-ap
cl-dual-ap
w-cl-dual-ap

flat: false


hub
w-hub
sec-hub
w-sec-hub
cl-hub
w-cl-hub
fe-hub
w-fe-hub
sec-fe-hub
w-sec-fe-hub
cl-fe-hub
w-cl-fe-hub
bridge
sec-bridge
bridge-ap
firewall
cl-firewall
sec-firewall
monitor
server
cloud
lock